Kruskal Algorithm

Group 3

03/2026

Định nghĩa

Kruskal là gì?

Kruskal là thuật toán tìm cây khung nhỏ nhất (Minimum Spanning Tree - MST) trong một đồ thị vô hướng, liên thông, có trọng số.

- Mục tiêu: nối tất cả các đỉnh với tổng trọng số nhỏ nhất
- Chọn các cạnh nhỏ nhất trước
- Không được tạo chu trình

Cây khung nhỏ nhất là gì?

- Cây khung là tập cạnh nối tất cả các đỉnh của đồ thị
- Có đúng $n - 1$ cạnh nếu đồ thị có n đỉnh
- Không có chu trình
- Tổng trọng số nhỏ nhất trong tất cả các cây khung

Kruskal giúp ta xây dựng MST bằng cách chọn cạnh tốt nhất từng bước

Ý tưởng của Kruskal

1. Sắp xếp tất cả các cạnh theo trọng số tăng dần
2. Duyệt từng cạnh từ nhỏ đến lớn
3. Nếu cạnh đó không tạo chu trình thì chọn
4. Nếu tạo chu trình thì bỏ qua
5. Dừng khi đã chọn đủ $n - 1$ cạnh

Chọn cạnh nhỏ nhất nhưng vẫn phải giữ đồ thị là một cây

Cấu trúc dữ liệu cần dùng

- Danh sách cạnh
- Sắp xếp cạnh theo trọng số
- Disjoint Set Union (DSU) / Union-Find

Vai trò của DSU

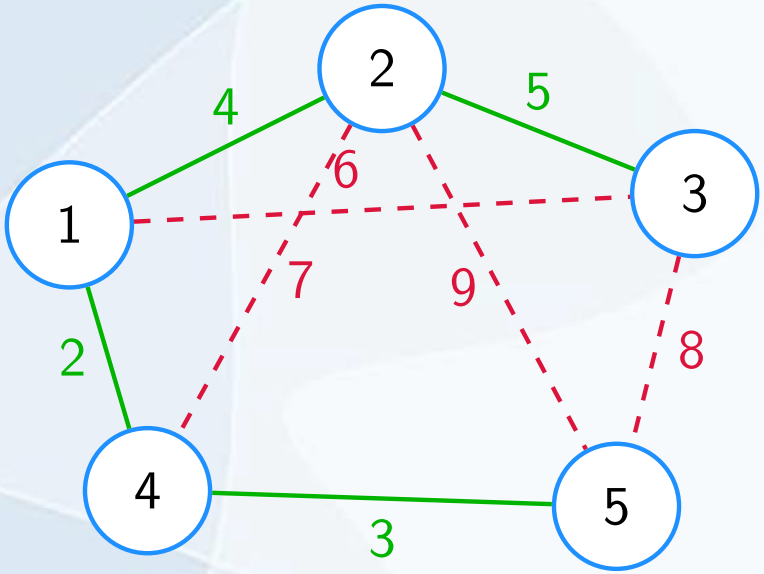
DSU dùng để kiểm tra nhanh hai đỉnh đang thuộc cùng một thành phần liên thông hay chưa. Nếu đã cùng thành phần, thêm cạnh vào sẽ tạo chu trình.

Disjoint Set Union

- `find(x)`: tìm gốc của tập chứa đỉnh x
- `union(a,b)`: gộp hai tập lại với nhau
- Dùng path compression và union by size/rank

Nếu $find(u) \neq find(v) \Rightarrow$ có thể chọn cạnh (u, v)

Minh họa trực quan



- Các cạnh màu xanh là cạnh được chọn
- Các cạnh nét đứt là cạnh bị loại
- Ta luôn xét cạnh từ nhỏ đến lớn

Quy trình chi tiết

1. Khởi tạo mỗi đỉnh là một tập riêng
2. Sắp xếp cạnh theo trọng số tăng dần
3. Lấy cạnh nhỏ nhất chưa xét
4. Kiểm tra hai đầu mút có thuộc cùng tập không
5. Nếu chưa:
   - thêm cạnh vào MST
   - gộp hai tập
6. Nếu rồi:
   - bỏ qua cạnh đó vì sẽ tạo chu trình

Ví dụ từng bước

Cạnh	Trọng số	Quyết định	Lý do
1–4	2	Chọn	Chưa tạo chu trình
4–5	3	Chọn	Chưa tạo chu trình
1–2	4	Chọn	Chưa tạo chu trình
2–3	5	Chọn	Đủ để nối thêm đỉnh
1–3	6	Bỏ	Tạo chu trình
2–4	7	Bỏ	Tạo chu trình

Sau khi chọn đủ $n - 1$ cạnh, ta thu được MST

Tính chất quan trọng

- Tham lam: luôn chọn cạnh nhỏ nhất có thể
- An toàn: cạnh được chọn không làm mất khả năng tối ưu
- Không tạo chu trình: điều kiện bắt buộc
- Đồ thị ban đầu phải liên thông để có MST

Vì sao Kruskal đúng?

- Khi xét cạnh theo thứ tự tăng dần, cạnh nhỏ hơn luôn được ưu tiên
- Nếu cạnh nối hai thành phần khác nhau, nó không thể làm xuất hiện chu trình
- Việc chọn cạnh nhỏ nhất an toàn giúp tổng trọng số toàn cây là nhỏ nhất

Ý tưởng cốt lõi: chọn cạnh nhẹ nhất mà vẫn giữ được tính cây

Code C++: Cài đặt trước

```
#include <bits/stdc++.h>
using namespace std;
struct Edge{
    int u, v;
    long long w;
};
int n, m, parent[MAXN], sz[MAXN];
vector<Edge> edges(m);
const int MAXN = 100005;
int find_set(int u){
    if(parent[u] == u) return u;
    return parent[u] = find_set(parent[u]);
}
```

Code C++: Hàm kiểm tra

```
bool unite(int u, int v){
    u = find_set(u);
    v = find_set(v);
    if(u == v) return false;
    if(sz[u] < sz[v]) swap(u, v);
    parent[v] = u;
    sz[u] += sz[v];
    return true;
}
```

Code C++: Khai báo

```
void khaibao(){
    cin >> n >> m;
    for(int i = 0; i < m; i++)
        cin >> edges[i].u >> edges[i].v >> edges[i].w;
    sort(edges.begin(), edges.end(), [](const Edge &a,
        const Edge &b){
        return a.w < b.w;
    });
    for(int i = 1; i <= n; i++){
        parent[i] = i;
        sz[i] = 1;
    }
}
```

Code C++: Trong hàm chính

```
int main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    long long mst_cost = 0;
    vector<Edge> chosen;
    for(auto &e : edges)
        if(unite(e.u, e.v)){
            mst_cost += e.w;
            chosen.push_back(e);
        }
    cout << mst_cost << '\n';
    for(auto &e : chosen)
        cout<<e.u <<' '<<e.v<<' '<<e.w<<'\n';
}
```

Độ phức tạp

- Sắp xếp m cạnh: $O(m \log m)$
- DSU:
  - `find`, `union` gần như $O(1)$ trung bình

Tổng độ phức tạp: $O(m \log m)$

Lỗi thường gặp

- Quên sắp xếp cạnh theo trọng số tăng dần
- Không dùng DSU nên kiểm tra chu trình chậm
- Không nén đường đi trong `find`
- Không dừng khi đã chọn đủ $n - 1$ cạnh
- Nhầm đồ thị có hướng với đồ thị vô hướng

Ứng dụng thực tế

- Thiết kế mạng điện, mạng đường, mạng máy tính
- Giảm chi phí kết nối các điểm
- Các bài toán phân cụm, tối ưu kết nối
- Xuất hiện rất nhiều trong lập trình thi đấu

Kết luận

- Kruskal là thuật toán cơ bản và rất quan trọng trong MST
- Ý tưởng đơn giản: chọn cạnh nhỏ nhất nhưng không tạo chu trình
- DSU là chìa khóa để cài đặt nhanh và đúng

Hiểu Kruskal là hiểu một trong những thuật toán greedy kinh điển nhất